

Universidad Mariano Gálvez de Guatemala

Centro regional Boca del Monte

Ingeniería en Sistemas de la Información

Curso: Análisis de Sistemas II

Ing. EZEQUIEL URIZAR ARAUJO



## Proyecto 1

Emanuel Feliciano Orozco Cifuentes 7690-23-6103

Lutwing Yussef Martinez Vasquez 7690-23-5854

Edson Rafael Gil Cardona 7690-23-20361

Julieta Ester Pinto Garcia 7690-24-742

**Guatemala de 27 de Agosto de 2026**

## Índice

|                                                                                                                                                                   |   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| E2 ARQUITECTURA .....                                                                                                                                             | 2 |
| 1. Decisión y justificación de la arquitectura .....                                                                                                              | 3 |
| Base de diseño .....                                                                                                                                              | 3 |
| 1.1. ....Atributos de calidad priorizados                                                                                                                         |   |
| 3                                                                                                                                                                 |   |
| 1.2. ....Estilo arquitectónico seleccionado                                                                                                                       |   |
| 4                                                                                                                                                                 |   |
| 1.3. .... Trade-offs asumidos                                                                                                                                     |   |
| 5                                                                                                                                                                 |   |
| 1.4. .... Monolito modular y descomposición                                                                                                                       |   |
| 5                                                                                                                                                                 |   |
| La separación permite que cada módulo mantenga una responsabilidad clara y evita que los cálculos financieros dependan directamente de componentes externos. .... | 6 |
| 1.5. .... Razones frente a la evolución futura                                                                                                                    |   |
| 6                                                                                                                                                                 |   |
| 1.6. ....Vista 4+1 de Kruchten                                                                                                                                    |   |
| 6                                                                                                                                                                 |   |
| 1.6.1. .... Vista lógica                                                                                                                                          |   |
| 7                                                                                                                                                                 |   |
| Componentes principales .....                                                                                                                                     | 7 |
| 1.6.2. ....Vista de desarrollo                                                                                                                                    |   |
| 8                                                                                                                                                                 |   |
| La vista de desarrollo representa la organización del código y la separación entre dominio, aplicación e infraestructura. ....                                    | 9 |
| La estructura implementada contempla principalmente:Figura 2: Vista de Desarrollo Estructura de carpetas .....                                                    | 9 |
| 1.6.3. ....Vista de procesos                                                                                                                                      |   |
| 9                                                                                                                                                                 |   |

|                                                                                                                                            |                                               |    |
|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|----|
| 1.6.4.....                                                                                                                                 | Vista de escenarios (+1)                      | 10 |
| 1.7.....                                                                                                                                   | Modelo C4                                     | 11 |
| 1.7.1.....                                                                                                                                 | Nivel 1 Contexto                              | 11 |
| 1.7.2.....                                                                                                                                 | Nivel 2 Contenedores                          | 12 |
| Figura 5: C4 Nivel 2 Contenedores .....                                                                                                    |                                               | 12 |
| 1.7.3.....                                                                                                                                 | Nivel 3 Componentes                           | 13 |
| 1.8.....                                                                                                                                   | Arquitectura hexagonal: puertos y adaptadores | 14 |
| Núcleo de dominio .....                                                                                                                    |                                               | 14 |
| Aplicación .....                                                                                                                           |                                               | 14 |
| E3 DISEÑO DE COMPONENTES .....                                                                                                             |                                               | 15 |
| 2. Correspondencia entre Diseño E3 y Código E4 .....                                                                                       |                                               | 15 |
| La correspondencia debe reflejar únicamente los archivos que realmente existen en el código. Correspondencia Diseño E3 con Código E4 ..... |                                               |    |
| 2.1.....                                                                                                                                   | Estado de cada componente                     | 16 |
| 2.2. Diseño detallado del módulo Cálculo Financiero .....                                                                                  |                                               | 17 |
| 2.2.1. Componente: Dinero .....                                                                                                            |                                               | 17 |
| 2.2.2. Componente: Estrategia de Amortización .....                                                                                        |                                               | 18 |
| 2.2.3. Componente: PlanAmortizacion.....                                                                                                   |                                               | 18 |
| 2.2.4. Componente: CalculadoraMora .....                                                                                                   |                                               | 19 |
| 2.2.5. Componente: PrelacionPago .....                                                                                                     |                                               | 19 |
| 2.2.6. Componente: Cartera en riesgo .....                                                                                                 |                                               | 20 |

|                                                                                                                                                                                                  |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.2.7. Componente: Estados del crédito .....                                                                                                                                                     | 21 |
| 2.2.8. Componente: RegistrarPago .....                                                                                                                                                           | 21 |
| 2.2.9. Componente: RepositorioPagos .....                                                                                                                                                        | 22 |
| 2.2.10. Componente: RepositorioPagosMemoria .....                                                                                                                                                | 22 |
| 2.3.Ciclo de vida con patrón State .....                                                                                                                                                         | 23 |
| 2.4. Principios SOLID aplicados .....                                                                                                                                                            | 24 |
| Los componentes futuros como IReloj, IGeneradorIds e IRepositorioCredito<br>deben considerarse Diseñados / No implementados cuando se presenten<br>como parte de la arquitectura propuesta. .... | 24 |
| 2.5. Principios GRASP aplicados.....                                                                                                                                                             | 24 |
| 2.6. Cohesión y acoplamiento por módulo .....                                                                                                                                                    | 25 |
| La separación permite modificar componentes externos sin alterar<br>directamente las reglas principales del dominio.....                                                                         | 25 |
| 2.7. Patrones de diseño aplicados.....                                                                                                                                                           | 25 |
| 2.7.1. Patrón Strategy (GoF).....                                                                                                                                                                | 26 |
| Solución .....                                                                                                                                                                                   | 26 |
| 2.7.2. Patrón State – Diseñado / No implementado .....                                                                                                                                           | 26 |
| 2.7.3. Patrón Chain of Responsibility – GoF.....                                                                                                                                                 | 27 |
| 2.7.4. Patrón Value Object – DDD .....                                                                                                                                                           | 28 |
| Estado .....                                                                                                                                                                                     | 29 |
| 2.7.5. Patrón Repository – Arquitectónico.....                                                                                                                                                   | 29 |
| 2.7.6. Builder – Diseñado / No implementado .....                                                                                                                                                | 30 |

## E2 ARQUITECTURA

### 1. Decisión y justificación de la arquitectura

#### Base de diseño

La arquitectura propuesta busca mantener separadas las reglas de negocio de los detalles de infraestructura. Esto es especialmente importante porque el sistema administra operaciones financieras en las que los cálculos deben ser exactos, reproducibles y trazables.

El diseño se organiza como un monolito modular con arquitectura hexagonal, donde el núcleo contiene las reglas de negocio y los componentes externos se conectan mediante puertos y adaptadores.

La solución se divide conceptualmente en los siguientes módulos:

- Origenación: registro de clientes, solicitudes, evaluación, aprobación y desembolso.
- Cálculo Financiero: representación de dinero, amortización y cálculo de mora.
- Cartera y Cobros: pagos, prelación de pagos y administración del ciclo de vida del crédito.
- Cierres: diseño destinado a la consolidación y congelamiento de información por período.
- Reportería: consultas y presentación de información de cartera.

En esta fase, el núcleo financiero es la parte implementada y comprobable mediante pruebas. Otros componentes se mantienen como diseño para futuras etapas.

#### 1.1. Atributos de calidad priorizados

La arquitectura se seleccionó considerando que el sistema trabaja con dinero y reglas financieras que deben generar resultados consistentes.

| Prioridad | Atributo de calidad    | Justificación                                                                                      |
|-----------|------------------------|----------------------------------------------------------------------------------------------------|
| 1         | Cumplimiento funcional | Las reglas de amortización, mora y prelación deben ejecutarse de acuerdo con las reglas definidas. |

|   |                          |                                                                                                             |
|---|--------------------------|-------------------------------------------------------------------------------------------------------------|
| 2 | Fiabilidad               | Los mismos datos de entrada deben producir los mismos resultados.                                           |
| 3 | Mantenibilidad           | Las reglas financieras pueden cambiar y deben poder evolucionar sin modificar todo el sistema.              |
| 4 | Seguridad / Trazabilidad | Las operaciones financieras deben poder ser registradas y verificadas.                                      |
| 5 | Eficiencia               | Los cálculos financieros se ejecutan como operaciones del núcleo sin depender de servicios externos.        |
| 6 | Evolución                | El diseño permite incorporar posteriormente API REST, Web, Chat, RAG y MCP.                                 |
| 7 | Portabilidad             | El núcleo desarrollado en TypeScript puede ejecutarse independientemente de una infraestructura específica. |
| 8 | Usabilidad               | Se considera principalmente para las interfaces que serán desarrolladas en etapas posteriores.              |

## 1.2. Estilo arquitectónico seleccionado

Se selecciona una arquitectura hexagonal dentro de un monolito modular.

El objetivo es mantener las reglas de negocio en un núcleo independiente de la infraestructura. De esta forma, los cálculos financieros no dependen directamente de una base de datos, de una interfaz gráfica o de un servidor HTTP.

La arquitectura se divide conceptualmente en:

1. Dominio: contiene las reglas y conceptos principales del negocio.

2. Aplicación: coordina los casos de uso.
3. Puertos: definen los contratos mediante los cuales el sistema se comunica con componentes externos.
4. Adaptadores: proporcionan implementaciones concretas para dichos contratos.

#### Comparación de alternativas:

| Alternativa                         | Ventaja                                              | Desventaja                                                   | Decisión                                |
|-------------------------------------|------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|
| <b>Arquitectura en capas</b>        | Fácil de comprender y ampliamente utilizada          | Puede mezclar responsabilidades                              | No adoptada                             |
| <b>Cliente-servidor</b>             | Separación entre cliente y servidor                  | Introduce dependencia de comunicación                        | No adoptada                             |
| <b>Microservicios</b>               | Escalado independiente                               | Mayor complejidad y consistencia distribuida                 | No adoptada                             |
| <b>Orientada a eventos</b>          | Buen desacoplamiento                                 | No es necesaria para las operaciones financieras principales | No adoptada como arquitectura principal |
| <b>Hexagonal + Monolito Modular</b> | Núcleo aislado, testeable y preparado para evolución | Requiere mantener límites claros                             | Seleccionada                            |

### 1.3. Trade-offs asumidos

La arquitectura seleccionada presenta los siguientes compromisos:

| Se obtiene                                   | Compromiso asumido                                                      |
|----------------------------------------------|-------------------------------------------------------------------------|
| Núcleo financiero aislado                    | Se deben definir contratos entre componentes.                           |
| Cálculos reproducibles                       | Se requiere controlar explícitamente tipos monetarios y redondeos.      |
| Facilidad para realizar pruebas              | Las funciones deben mantenerse independientes de infraestructura.       |
| Posibilidad de incorporar nuevos adaptadores | Algunos componentes permanecen diseñados pero todavía no implementados. |

#### 1.4. Monolito modular y descomposición

El sistema se plantea como una única aplicación con fronteras internas bien definidas.

Esta decisión permite evitar la complejidad de una arquitectura distribuida mientras el proyecto todavía se encuentra en una etapa de desarrollo inicial.

| Módulo             | Responsabilidad Única                                                 |
|--------------------|-----------------------------------------------------------------------|
| Originación        | Gestionar clientes, solicitudes, evaluación, aprobación y desembolso. |
| Cálculo Financiero | Manejar dinero, amortización y cálculo de mora.                       |
| Cartera y Cobros   | Gestionar pagos, prelación y operaciones relacionadas con la cartera. |
| Cierres            | Consolidar información por período. Diseñado / no implementado en E4. |
| Reportería         | Consultar y presentar información financiera.                         |

La separación permite que cada módulo mantenga una responsabilidad clara y evita que los cálculos financieros dependan directamente de componentes externos.

#### 1.5. Razones frente a la evolución futura

La arquitectura permite agregar posteriormente diferentes adaptadores sin modificar las reglas principales del dominio.

Entre los componentes previstos se encuentran:

- API REST.
- Interfaz Web.
- PostgreSQL.
- Chat.
- RAG.
- Servidor MCP.

Estos elementos forman parte de la evolución prevista del sistema y no deben considerarse implementados en E4.

El Chat y el servidor MCP se mantienen dentro del modelo C4 como componentes de integración futura, debido a que el enunciado requiere mostrar su conexión con el sistema.



## **1.6. Vista 4+1 de Kruchten**

La arquitectura se documenta utilizando el modelo 4+1 de Kruchten, mediante las siguientes vistas:

1. Vista lógica.
2. Vista de desarrollo.
3. Vista de procesos.
4. Vista de escenarios.
5. Escenarios como elemento integrador.

### **1.6.1. Vista lógica**

La vista lógica representa los principales componentes y responsabilidades del sistema.

Componentes principales

Originación

Responsable de los procesos relacionados con el inicio del ciclo del crédito.

Cálculo Financiero

Contiene las operaciones relacionadas con:

- Dinero.
- Estrategias de amortización.
- Plan de amortización.
- Cálculo de mora.

### **Cartera y Cobros**

Contiene las operaciones relacionadas con:

- Registro de pagos.
- Aplicación de prelación.
- Administración de la cartera.
- Estados del crédito.

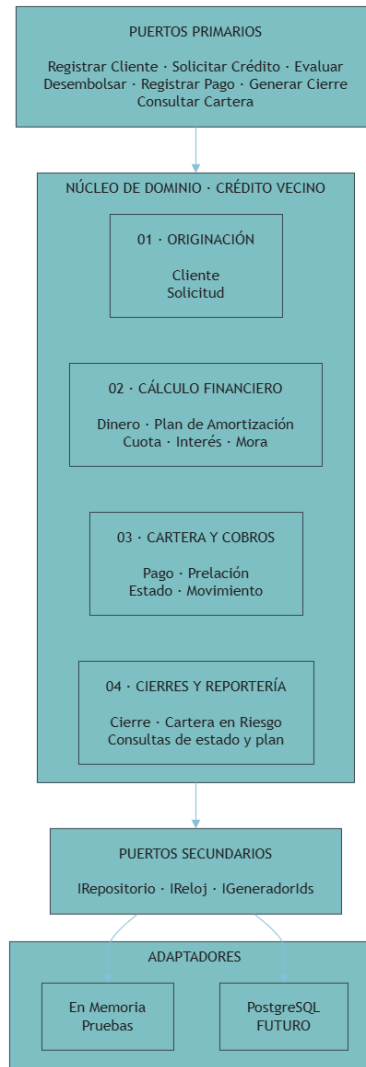
### **Cierres**

Componente diseñado para consolidar información por fecha de corte. Actualmente se encuentra diseñado/no implementado.

## Reportería

Componente destinado a consultar información de cartera y resultados financieros.

**Figura 1: Vista Lógica del Sistema**

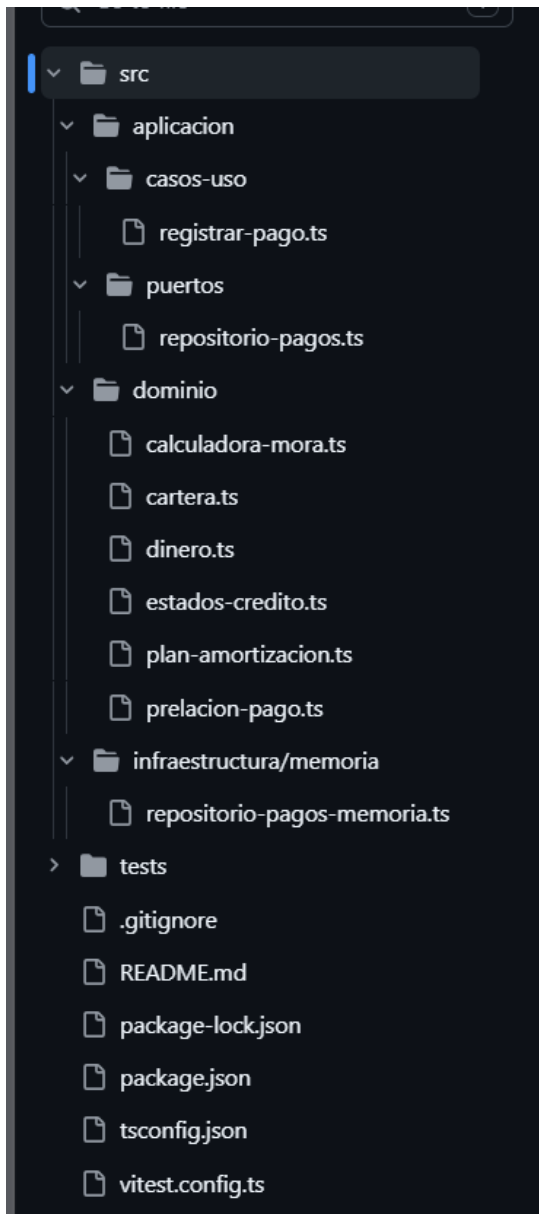


### 1.6.2. Vista de desarrollo

La vista de desarrollo representa la organización del código y la separación entre dominio, aplicación e infraestructura.

La estructura implementada contempla principalmente:

**Figura 2: Vista de Desarrollo Estructura de carpetas**



### 1.6.3. Vista de procesos

La vista de procesos representa la ejecución de una operación dentro del sistema.

Como escenario representativo se utiliza Registrar Pago.

El proceso involucra:

1. Recepción del pago.
2. Consulta del pago o crédito correspondiente.
3. Identificación de la deuda.
4. Cálculo de mora cuando corresponde.
5. Aplicación de la prelación de pago.
6. Actualización de la información.
7. Verificación del estado resultante.

El diseño busca que las operaciones financieras sean realizadas por componentes especializados y que el caso de uso coordine las operaciones sin asumir directamente las reglas financieras.

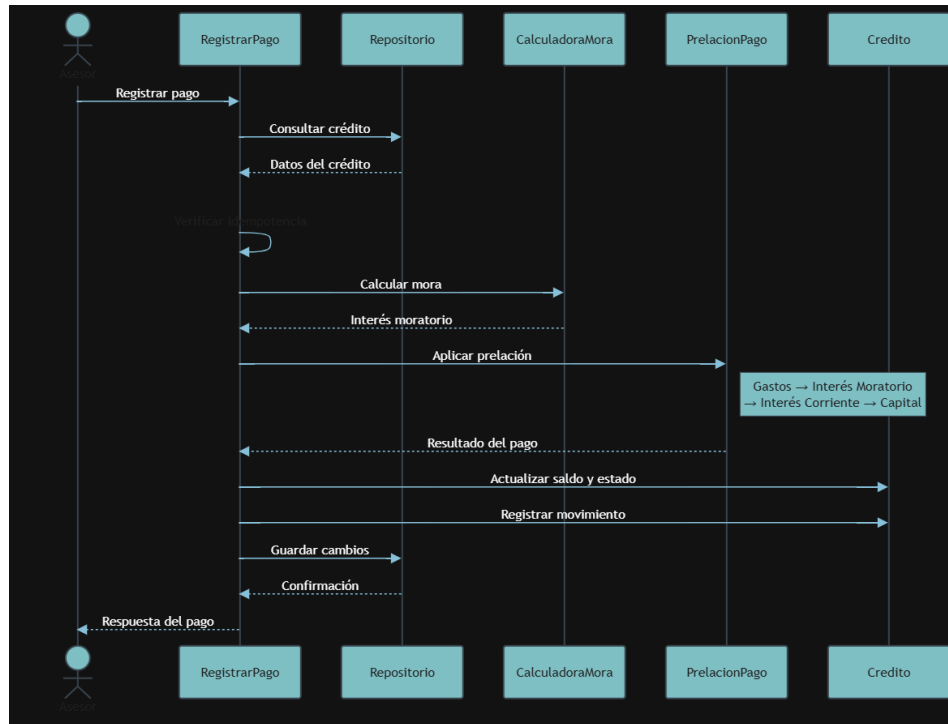
#### **1.6.4. Vista de escenarios (+1)**

La vista de escenarios relaciona los componentes de la arquitectura con situaciones concretas de uso.

El escenario seleccionado es Registrar Pago, debido a que permite demostrar la interacción entre:

- RegistrarPago.
- RepositorioPagos.
- CalculadoraMora.
- PrelacionPago.
- Estados del crédito.

#### **Figura 3. Vista de escenarios — Diseño objetivo de Registrar Pago**



La implementación E4 valida actualmente la idempotencia y persistencia en memoria. La coordinación completa con mora, prelación, saldo y movimientos permanece como diseño arquitectónico.

## 1.7. Modelo C4

El modelo C4 se utiliza para representar la arquitectura desde diferentes niveles de detalle.

### 1.7.1. Nivel 1 Contexto

El sistema interactúa principalmente con:

- Cliente.
- Asesor de crédito.
- Comité de crédito.
- Gerencia.

También se contemplan futuras integraciones con:

- Chat.
- RAG.
- Servidor MCP

**Figura 4: C4 Nivel 1 — Contexto del Sistema**



### 1.7.2. Nivel 2 Contenedores

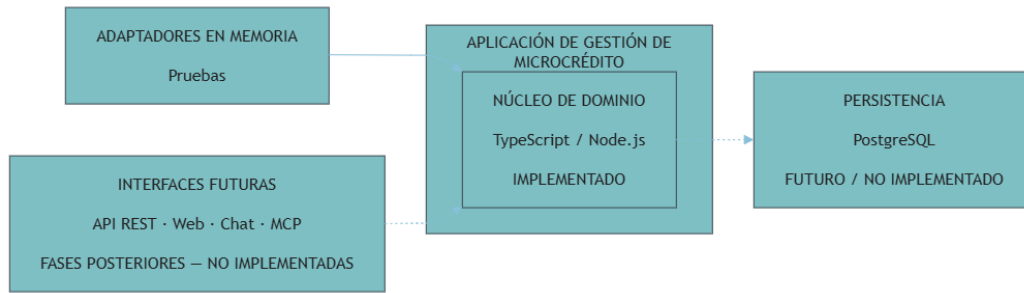
Los principales contenedores conceptuales son:

- Sistema de Gestión de Créditos.
- Núcleo de dominio.
- Aplicación.
- Adaptadores de infraestructura.

Como elementos futuros se consideran:

- API REST.
- Interfaz Web.
- PostgreSQL.
- Chat.
- RAG.
- Servidor MCP.

**Figura 5: C4 Nivel 2 Contenedores**



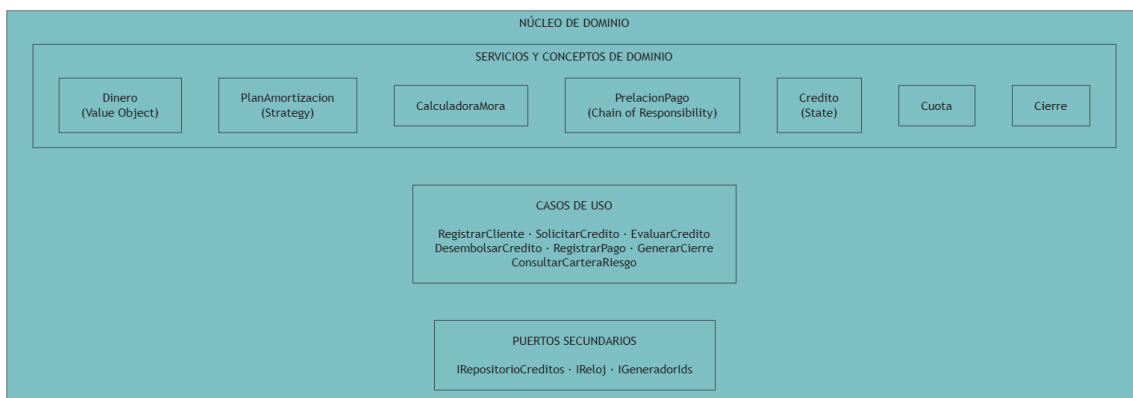
### 1.7.3. Nivel 3 Componentes

Dentro del núcleo se identifican componentes como:

- Dinero.
- EstrategiaAmortizacion.
- AmortizacionFrancesa.
- PlanAmortizacion.
- CalculadoraMora.
- PrelacionPago.
- EslabonPrelacion.
- Cartera.
- EstadosCredito.
- RegistrarPago.
- RepositorioPagos.

El repositorio en memoria funciona como adaptador para pruebas.

**Figura 6: C4 Nivel 3 Componentes del Núcleo**



## **1.8. Arquitectura hexagonal: puertos y adaptadores**

La arquitectura se organiza alrededor del núcleo del dominio.

### **Núcleo de dominio**

Contiene las reglas financieras y componentes que no dependen directamente de infraestructura.

Entre ellos:

- Dinero.
- PlanAmortizacion.
- CalculadoraMora.
- PrelacionPago.
- Cartera.
- EstadosCredito.

### **Aplicación**

Coordina los casos de uso.

Un ejemplo implementado es:

- RegistrarPago.

### **Puertos secundarios**

Permiten que la aplicación se comuniquen con elementos externos sin depender de una implementación concreta.

El puerto actualmente utilizado para pagos es:

- RepositorioPagos.

### **Adaptadores**

Actualmente existe:

- RepositorioPagosMemoria — utilizado para pruebas.

Se contempla posteriormente:



- RepositorioPagosPostgreSQL — futuro/no implementado.

## E3 DISEÑO DE COMPONENTES

### 2. Correspondencia entre Diseño E3 y Código E4

La correspondencia debe reflejar únicamente los archivos que realmente existen en el código. Correspondencia Diseño E3 con Código E4

| Diseño en E3               | Archivo real                                             | Estado                           |
|----------------------------|----------------------------------------------------------|----------------------------------|
| Dinero                     | src/dominio/dinero.ts                                    | Implementado                     |
| EstrategiaAmortizacion     | src/dominio/plan-amortizacion.ts                         | Implementado                     |
| AmortizacionFrancesa       | src/dominio/plan-amortizacion.ts                         | Implementado                     |
| PlanAmortizacion           | src/dominio/plan-amortizacion.ts                         | Implementado                     |
| Cálculo de Mora            | src/dominio/calculadora-mora.ts                          | Implementado                     |
| Prelacion de Pago          | src/dominio/prelacion-pago.ts                            | Implementado                     |
| Cartera en riesgo          | src/dominio/cartera.ts                                   | Implementado                     |
| Estado del Crédito         | src/dominio/estados-credito.ts                           | Implementado                     |
| RegistrarPago              | src/aplicacion/casos-uso/registrar-pago.ts               | Implementado                     |
| RepositorioPagos           | src/aplicacion/puertos/repositorio-pagos.ts              | Implementado                     |
| RepositorioPagosMemoria    | src/infraestructura/memoria/repositorio-pagos-memoria.ts | Implementado — Solo para pruebas |
| RepositorioPagosPostgreSQL | No implementado                                          | Diseñado / No implementado       |
| Cierre                     | No implementado                                          | Diseñado / No implementado       |
| IRepositorioCreditos       | No implementado                                          | Diseñado / No implementado       |

|               |                 |                            |
|---------------|-----------------|----------------------------|
| IReloj        | No implementado | Diseñado / No implementado |
| IGeneradorIds | No implementado | Diseñado / No implementado |
| API REST      | No implementado | Diseñado / No implementado |
| Interfaz Web  | No implementado | Diseñado / No implementado |
| Chat          | No implementado | Diseñado / No implementado |
| RAG           | No implementado | Diseñado / No implementado |
| Servidor MCP  | No implementado | Diseñado / No implementado |

## 2.1. Estado de cada componente

Los componentes implementados actualmente se concentran principalmente en el núcleo financiero y en el caso de uso relacionado con el registro de pagos.

| Componente              | Estado                           |
|-------------------------|----------------------------------|
| Dinero                  | Implementado                     |
| EstrategiaAmortizacion  | Implementado                     |
| PlanAmortizacion        | Implementado                     |
| CalculadoraMora         | Implementado                     |
| PrelacionPago           | Implementado                     |
| Cartera                 | Implementado                     |
| EstadosCredito          | Implementado                     |
| RegistrarPago           | Implementado                     |
| RepositorioPagos        | Implementado                     |
| RepositorioPagosMemoria | Implementado — Solo para pruebas |
| Cierre                  | Diseñado — No implementado       |
| IRepositorioCreditos    | Diseñado — No implementado       |

|                            |                            |
|----------------------------|----------------------------|
| IReloj                     | Diseñado — No implementado |
| IGeneradorIds              | Diseñado — No implementado |
| RepositorioPagosPostgreSQL | Diseñado — No implementado |
| API REST                   | Diseñado — No implementado |
| Interfaz Web               | Diseñado — No implementado |
| Chat                       | Diseñado — No implementado |
| RAG                        | Diseñado — No implementado |
| Servidor MCP               | Diseñado — No implementado |

## 2.2. Diseño detallado del módulo Cálculo Financiero

El módulo de Cálculo Financiero contiene las reglas relacionadas con las operaciones monetarias y financieras.

Su objetivo es mantener los cálculos separados de infraestructura y permitir que puedan probarse de manera independiente.

### 2.2.1. Componente: Dinero

**Responsabilidad:** representar valores monetarios de forma segura e inmutable.

El componente utiliza Decimal de la biblioteca decimal.js para evitar problemas derivados de la representación de números de punto flotante.

#### Interfaz pública real

- desde()
- cero()
- sumar()
- restar()
- multiplicar()
- menorQue()
- menorOIgualQue()
- esIgualA()
- esCero()
- esNegativo()
- comoDecimal()
- comoTexto()

#### Tipo interno

Decimal de decimal.js

#### Patrón

## **Value Object — DDD**

### **Archivo**

src/dominio/dinero.ts

### **2.2.2. Componente: Estrategia de Amortización**

La estrategia de amortización permite separar el algoritmo de cálculo del plan de amortización.

### **Responsabilidad**

Definir el comportamiento utilizado para generar el plan de amortización.

### **Implementación actual**

#### **AmortizacionFrancesa**

La estrategia genera las cuotas utilizando el método de amortización francés.

### **Archivo**

src/dominio/plan-amortizacion.ts

### **Patrón**

## **Strategy — GoF**

### **2.2.3. Componente: PlanAmortizacion**

### **Responsabilidad**

Representar el resultado del cálculo de amortización y las cuotas correspondientes al crédito.

El plan utiliza la estrategia de amortización definida para generar las cuotas.

### **Archivo**

src/dominio/plan-amortizacion.ts

### **Estado**

### **Implementado**

### **2.2.4. Componente: CalculadoraMora**

La calculadora de mora se implementa como un **servicio de dominio funcional**.

No debe representarse como una clase que contenga métodos como `calcularMoratorio()` o `clasificarTramo()`.

### **Funciones reales**

- `calcularDiasAtraso()`
- `clasificarMora()`
- `calcularMora()`

### **Responsabilidad**

Determinar los días de atraso, clasificar la mora y calcular el interés moratorio correspondiente.

El cálculo considera únicamente el capital que se encuentra en mora.

### **Archivo**

`src/dominio/calculadora-mora.ts`

### **Estado**

### **Implementado**

#### **2.2.5. Componente: PrelacionPago**

La prelación de pagos determina el orden en que un pago debe aplicarse sobre las obligaciones pendientes.

### **Orden de aplicación**

1. Gastos.
2. Interés moratorio.
3. Interés corriente.
4. Capital.

### **Interfaz real**

`aplicarPrelacion(pago, deuda): AplicacionPago`

La implementación utiliza una cadena de responsabilidad mediante la clase:

EslabonPrelacion

Cada eslabón representa un concepto de la deuda y procesa la parte correspondiente antes de trasladar el remanente al siguiente eslabón.

### **Cadena**

Gastos



InteresMoratorio



InteresCorriente



Capital

### **Archivo**

src/dominio/prelacion-pago.ts

### **Patrón**

## **Chain of Responsibility — GoF**

### **2.2.6. Componente: Cartera en riesgo**

La cartera en riesgo se calcula mediante una función del dominio.

### **Función real**

calcularCarteraRiesgo(creditos): ResultadoCarteraRiesgo

### **Responsabilidad**

Determinar el capital correspondiente a créditos que cumplen las condiciones establecidas para considerarse en riesgo.

El componente permite obtener información relacionada con:

- Capital en riesgo.
- Total de cartera.
- Porcentaje de cartera en riesgo.

### **Archivo**

src/dominio/cartera.ts

### **Estado**

### **Implementado**

### 2.2.7. Componente: Estados del crédito

El código actual no implementa todavía un conjunto de clases polimórficas como:

- EstadoVigente.
- EstadoEnMora.
- EstadoCancelado.
- EstadoIncobrable.

Por lo tanto, el documento no debe afirmar que esas clases forman parte actualmente de E4.

El código actual trabaja con:

- EstadoCredito
- SituacionCredito
- situacion()
- actualizarPorPago()
- actualizarPorCorte()

### Descripción actual

Las transiciones del crédito se encuentran implementadas mediante **funciones puras**.

El diseño polimórfico mediante el patrón **State** representa una evolución propuesta para el sistema, pero todavía no corresponde a una implementación completa del patrón en E4.

### Estado

### Implementado mediante funciones puras / diseño State propuesto

### 2.2.8. Componente: RegistrarPago

Representa el caso de uso encargado de coordinar el registro de un pago.

### Responsabilidad

- Registrar un pago de manera idempotente, evitar duplicados y rechazar la reutilización de una clave con datos diferentes. Validar el comando.
- Buscar por clave de idempotencia.

- Validar el reintento.
- Guardar el pago.

El flujo completo con cálculo de mora, prelación y actualización del crédito corresponde al diseño arquitectónico objetivo, pero todavía no está integrado dentro del caso de uso RegistrarPago.

### **Archivo**

src/aplicacion/casos-uso/regarstrar-pago.ts

### **Estado**

### **Implementado**

## **2.2.9. Componente: RepositorioPagos**

Representa el puerto utilizado para abstraer la persistencia de pagos.

### **Operaciones**

- buscarPorClave()
- guardar()

### **Archivo**

src/aplicacion/puertos/repositorio-pagos.ts

### **Estado**

### **Implementado**

## **2.2.10. Componente: RepositorioPagosMemoria**

Es el adaptador utilizado para proporcionar una implementación en memoria del repositorio de pagos.

### **Operaciones**

- buscarPorClave()
- guardar()
- cantidad()

### **Archivo**



src/infraestructura/memoria/repositorio-pagos-memoria.ts  
Estado

## Implementado — Solo para pruebas

### 2.3.Ciclo de vida con patrón State

El ciclo de vida del crédito se encuentra contemplado en el diseño del sistema.

Sin embargo, debe distinguirse entre la implementación actual y el diseño polimórfico propuesto.

Actualmente, el código utiliza:

- EstadoCredito
- SituacionCredito
- situacion()
- actualizarPorPago()
- actualizarPorCorte()

Las transiciones se encuentran implementadas mediante funciones puras.

El diseño polimórfico mediante State representa una evolución propuesta para el sistema.

### Clasificación de mora

Los tramos de mora se consideran una clasificación derivada y no estados del crédito.

| Tramo      | Días de atraso  |
|------------|-----------------|
| Mora 1     | 1–30 días       |
| Mora 2     | 31–60 días      |
| Mora 3     | 61–90 días      |
| Vencido    | 91–120 días     |
| Incobrable | Más de 120 días |

### 2.4. Principios SOLID aplicados

| Principio                   | Aplicación en el Sistema                                                             |
|-----------------------------|--------------------------------------------------------------------------------------|
| SRP — Responsabilidad Única | Dinero concentra operaciones monetarias; CalculadoraMora concentra cálculos de mora; |

|                                 |                                                                                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
|                                 | PrelacionPago administra la distribución de pagos.                                                                                        |
| OCP — Abierto/Cerrado           | La estrategia de amortización permite incorporar diferentes métodos mediante un contrato común.                                           |
| LSP — Sustitución               | Las implementaciones que cumplen contratos definidos pueden sustituirse sin modificar al consumidor.                                      |
| ISP — Interfaces específicas    | Los contratos se mantienen pequeños y orientados a responsabilidades específicas.                                                         |
| DIP — Inversión de dependencias | La aplicación utiliza contratos para comunicarse con componentes externos, evitando depender directamente de una implementación concreta. |

Los componentes futuros como IReloj, IGeneradorIds e IRepositorioCreditos deben considerarse Diseñados / No implementados cuando se presenten como parte de la arquitectura propuesta.

## 2.5. Principios GRASP aplicados

| Principio         | Aplicación                                                                                                          |
|-------------------|---------------------------------------------------------------------------------------------------------------------|
| Experto           | Los componentes del dominio mantienen las responsabilidades relacionadas con la información que conocen.            |
| Creador           | Los componentes encargados de generar planes y resultados financieros concentran la construcción de dichos objetos. |
| Controlador       | Los casos de uso coordinan las solicitudes y delegan la lógica correspondiente al dominio.                          |
| Bajo Acoplamiento | Los módulos se comunican mediante contratos y evitan depender directamente de infraestructura.                      |
| Alta Cohesión     | Cada componente mantiene responsabilidades relacionadas y específicas.                                              |
| Polimorfismo      | Se contempla mediante el diseño State y las estrategias de amortización.                                            |
| Indirección       | Los puertos permiten separar el dominio de los detalles de infraestructura.                                         |

|                        |                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------|
| Variaciones Protegidas | Las partes que pueden cambiar, como estrategias y persistencia, se protegen mediante contratos. |
|------------------------|-------------------------------------------------------------------------------------------------|

## 2.6. Cohesión y acoplamiento por módulo

| Módulo             | Cohesión | Acoplamiento | Justificación                                                                            |
|--------------------|----------|--------------|------------------------------------------------------------------------------------------|
| Originación        | Alta     | Bajo         | Agrupar las operaciones relacionadas con el inicio del crédito.                          |
| Cálculo Financiero | Muy Alta | Muy Bajo     | Contiene operaciones financieras independientes de infraestructura.                      |
| Cartera y Cobros   | Alta     | Bajo         | Coordina pagos y operaciones relacionadas con créditos.                                  |
| Cierres            | Alta     | Bajo         | Representa responsabilidades específicas de consolidación; actualmente es diseño futuro. |
| Reportería         | Alta     | Bajo         | Se orienta principalmente a consulta de información.                                     |

La separación permite modificar componentes externos sin alterar directamente las reglas principales del dominio.

## **2.7. Patrones de diseño aplicados**

Actualmente se consideran cuatro patrones implementados o aplicados en el código:

1. Strategy.
2. Chain of Responsibility.
3. Value Object.
4. Repository.

Los patrones State y Builder se mantienen como propuestas de diseño y no deben contarse como implementados en E4.

### **2.7.1. Patrón Strategy (GoF)**

#### **Problema**

La generación de planes de amortización puede utilizar diferentes métodos y políticas.

#### **Solución**

Se encapsula el algoritmo de amortización mediante una estrategia que permite intercambiar el comportamiento.

#### **Aplicación**

El diseño contempla:

- EstrategiaAmortizacion.
- AmortizacionFrancesa.
- PlanAmortizacion.

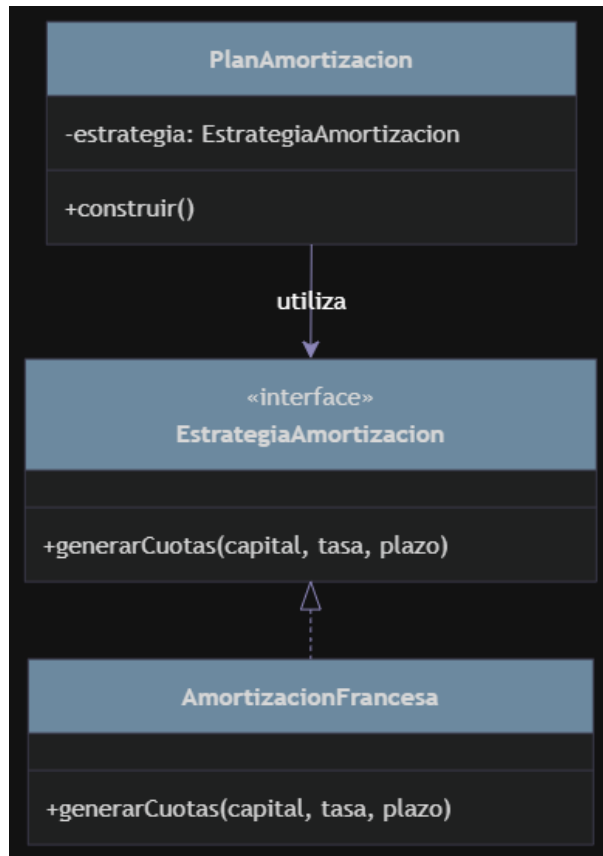
#### **Archivo**

src/dominio/plan-amortizacion.ts

#### **Estado**

Implementado.

### **Figura 7. Patrón Strategy – Amortización**



### 2.7.2. Patrón State – Diseñado / No implementado

#### Problema

El crédito posee diferentes situaciones durante su ciclo de vida y determinadas operaciones dependen de la situación actual.

#### Diseño propuesto

El patrón State propone representar cada estado mediante un comportamiento especializado.

Sin embargo, esta representación polimórfica todavía no corresponde directamente con el código actual.

#### Implementación actual

El código utiliza:

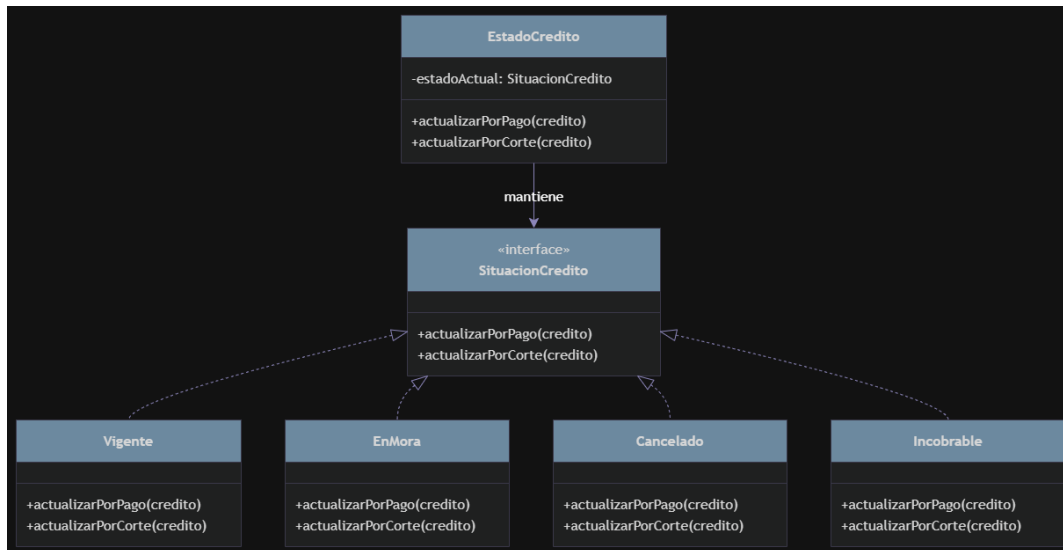
- EstadoCredito.
- SituacionCredito.
- situacion().

- actualizarPorPago().
- actualizarPorCorte().

Por lo tanto:

**Estado: Diseñado / No implementado como State polimórfico.**

**Figura 8. Patrón State – Diseño propuesto**



### 2.7.3. Patrón Chain of Responsibility – GoF

#### Problema

Un pago debe aplicarse siguiendo un orden específico.

#### Solución

Se utiliza una cadena de responsabilidades donde cada elemento procesa la parte correspondiente y pasa el remanente al siguiente.

#### Orden de prelación

**Gastos → Interés Moratorio → Interés Corriente → Capital**

#### Componente principal

EslabonPrelacion

#### Operación

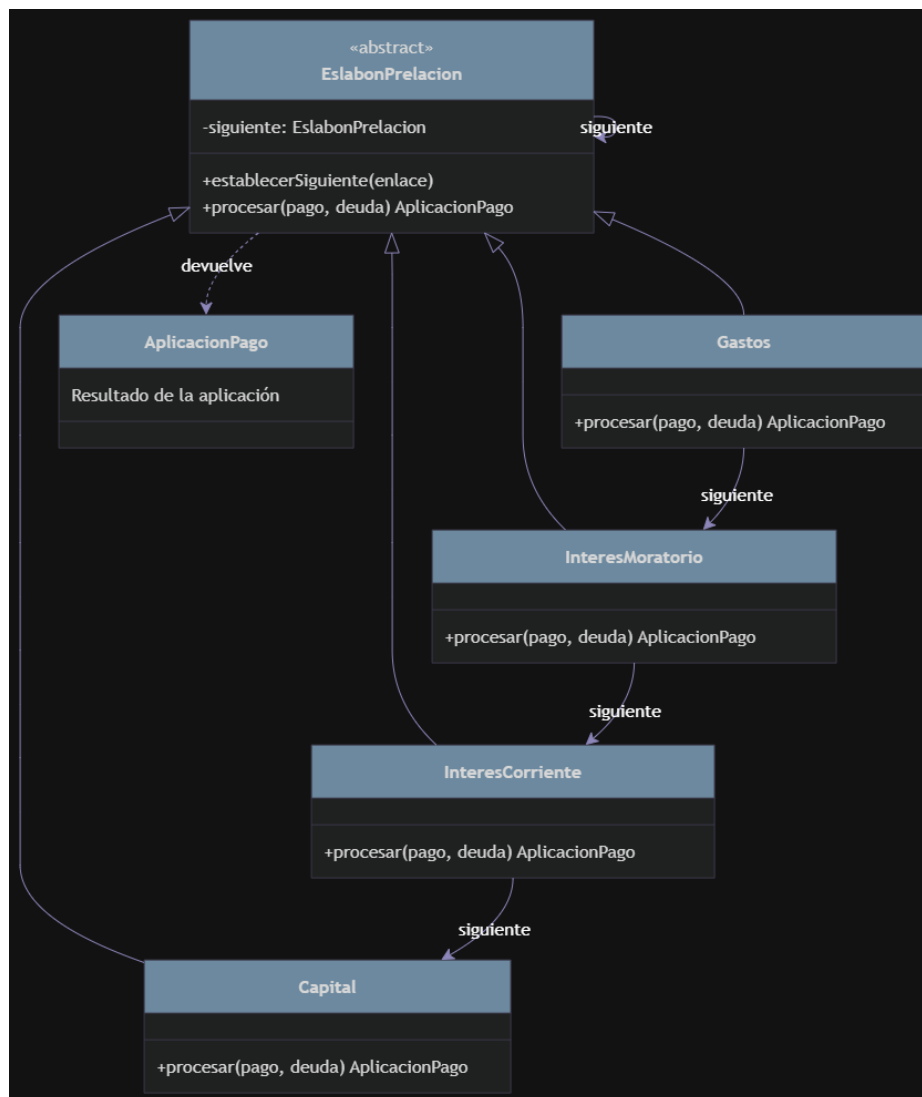
aplicarPrelacion(pago, deuda): AplicacionPago  
**Archivo**

src/dominio/prelacion-pago.ts

**Estado**

Implementado.

**Figura 9. Patrón Chain of Responsibility – Prelación de pago**



## 2.7.4. Patrón Value Object – DDD

### Problema

Las operaciones financieras requieren una representación controlada de los valores monetarios.

## **Solución**

Se utiliza Dinero como objeto de valor inmutable para encapsular las operaciones monetarias.

## **Archivo**

src/dominio/dinero.ts

## **Implementación interna**

Se utiliza:

Decimal **de** decimal.js

## **Operaciones principales**

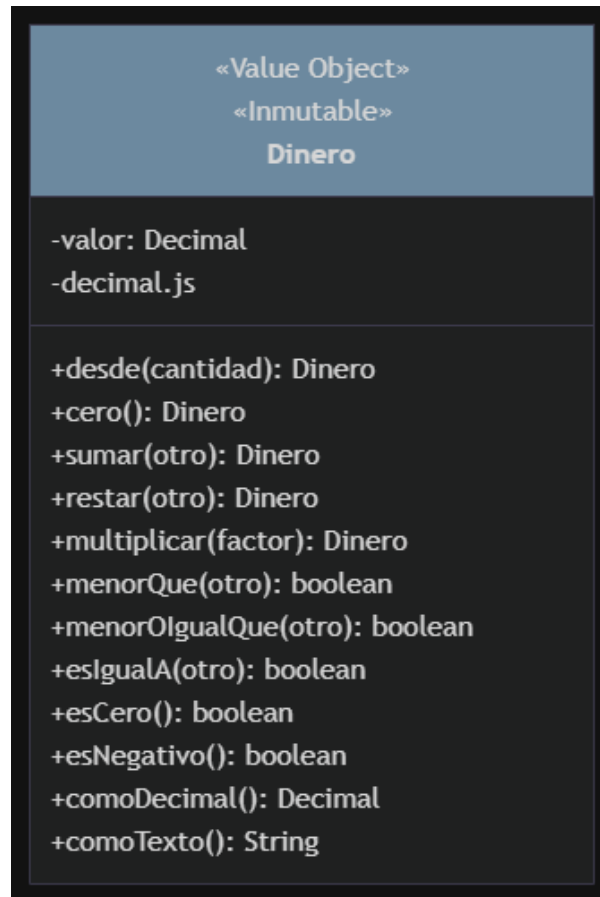
- desde()
- cero()
- sumar()
- restar()
- multiplicar()
- menorQue()
- menorIgualQue()
- esIgualA()
- esCero()
- esNegativo()
- comoDecimal()
- comoTexto()

Estado

Implementado.

## **Figura 10. Patrón Value Object – Dinero**





## 2.7.5. Patrón Repository – Arquitectónico

### Problema

El dominio y los casos de uso no deben depender directamente de una tecnología específica de almacenamiento.

### Solución

Se define un contrato RepositorioPagos y una implementación concreta RepositorioPagosMemoria.

### Componentes

#### RepositorioPagos

src/aplicacion/puertos/repositorio-pagos.ts

Operaciones:

- buscarPorClave()

- guardar()

## RepositorioPagosMemoria

src/infraestructura/memoria/repositorio-pagos-memoria.ts

- buscarPorClave()
- guardar()
- Cantidad()

Estado:

**Implementado – Solo para pruebas.**

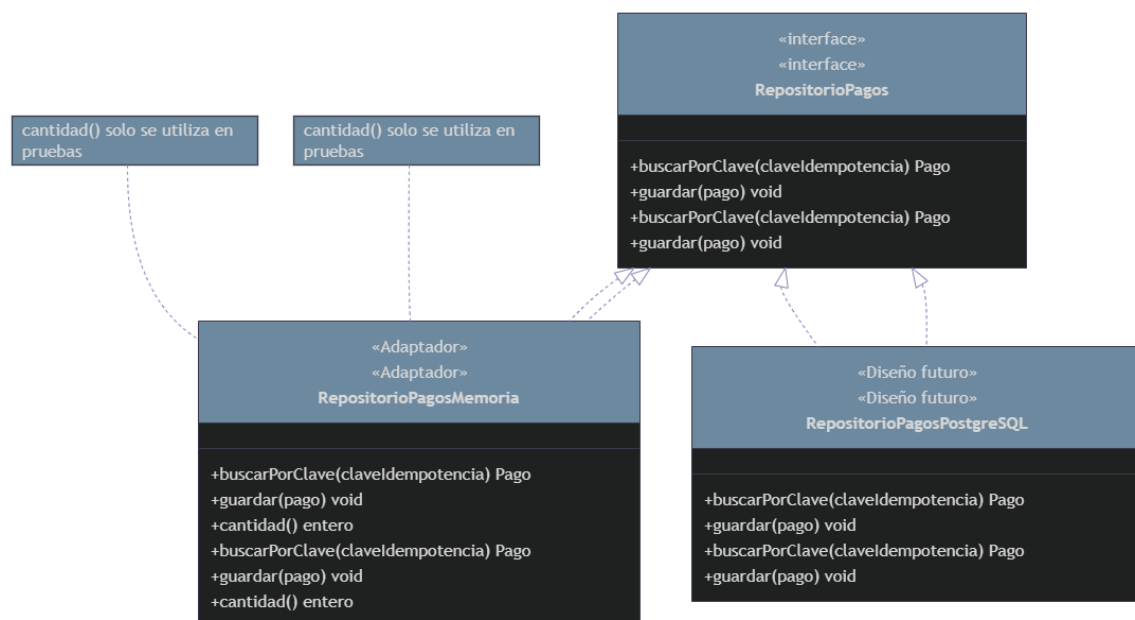
## PostgreSQL

RepositorioPagosPostgreSQL

Estado:

Diseñado / No implementado.

**Figura 11. Patrón Repository – Persistencia de pagos**



## 2.7.6. Builder – Diseñado / No implementado

El patrón Builder se mantiene como una propuesta de diseño para una construcción controlada de objetos relacionados con el plan de amortización.

Sin embargo, PlanAmortizacionBuilder no se encuentra implementado actualmente en E4.

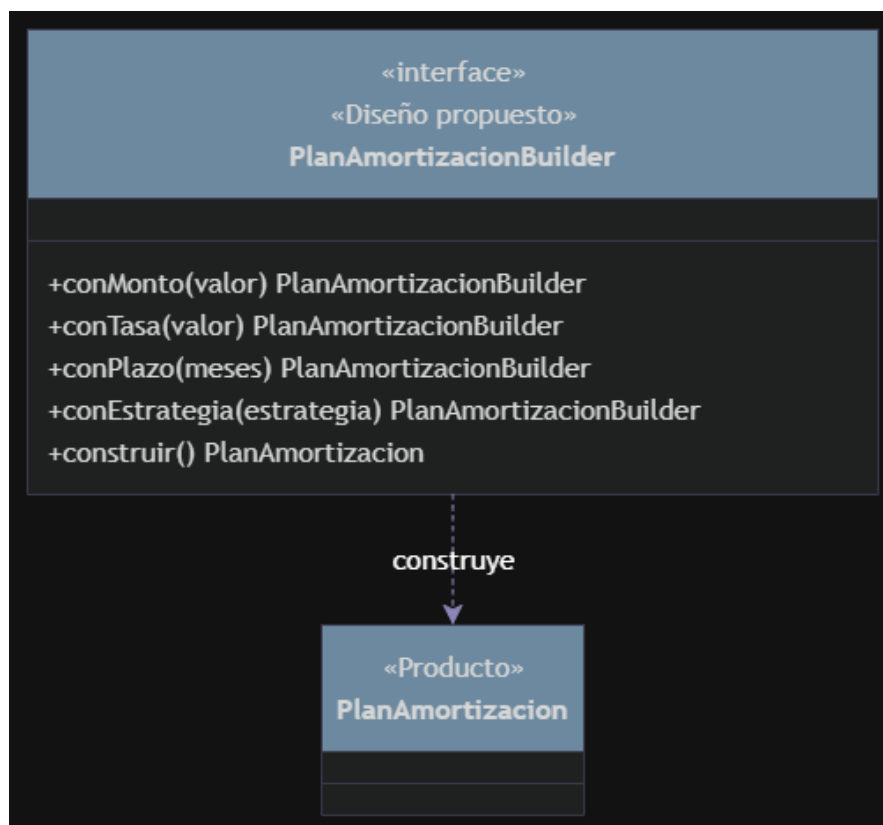
Por esta razón, Builder:

- No forma parte de los patrones implementados.
- No debe contarse dentro de los cuatro patrones actualmente aplicados.
- Puede mantenerse como una propuesta para una evolución posterior.

## Estado

**Diseñado / No implementado.**

**Figura 12. Builder – Diseño propuesto**



## 2.8. Componentes futuros y evolución de la arquitectura

Para evitar confundir el diseño arquitectónico con la implementación actual, los siguientes elementos se clasifican explícitamente como Diseñados / No implementados:

| <b>Componente</b>          | <b>Estado</b>              |
|----------------------------|----------------------------|
| Cartera                    | En riesgo - Implementado   |
| Cierre                     | Diseñado / No implementado |
| IRepositorioCreditos       | Diseñado / No implementado |
| IReloj                     | Diseñado / No implementado |
| IGeneradorIds              | Diseñado / No implementado |
| Cierre                     | Diseñado / No implementado |
| RepositorioPagosPostgreSQL | Diseñado / No implementado |
| API REST                   | Diseñado / No implementado |
| Interfaz Web               | Diseñado / No implementado |
| Chat                       | Diseñado / No implementado |
| RAG                        | Diseñado / No implementado |
| Servidor MCP               | Diseñado / No implementado |
| PlanAmortizacionBuilder    | Diseñado / No implementado |
| State polimórfico          | Diseñado / No implementado |